Image 1: My model code

```
const mongoose = require('mongoose');
const Schema = mongoose.Schema;

const TaskSchema = new Schema({
    name: {
        type: String,
        required: 'Kindly enter the name of the task'
    },
    created_date: {
        type: Date,
        default: Date.now
    },
    status: {
        type: [{
          type: String,
          enum: ['pending', 'ongoing', 'completed']
        }],
        default: ['pending']
    },
    made_by: {
        type: String,
        required: 'Kindly enter the name of the user who created the task',
        default: 'Nguyen Tan Huy - GCS230465'
    }
});

module.exports = mongoose.model('Tasks', TaskSchema);
```

Image 2: My server running on with Nodemon on Port 3000

```
PS D:\npmtest\todoListApi> npm run start
[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
Nguyen Tan Huy - todo list RESTful API server started on: 3000
(node:6824) [MONGOOSE] DeprecationWarning: Mongoose: the `strictQuery` option will be switched back to `false` by default in Mongoose 7. Use `mongoos
e.set('strictQuery', false);` if you want to prepare for this change. Or use `mongoose.set('strictQuery', true);` to suppress this warning.
(Use `node --trace-deprecation ...` to show where the warning was created)
```

Image 3: My API connected in Postcode showing some data from my database

GET http://localhost:3000/tasks Send

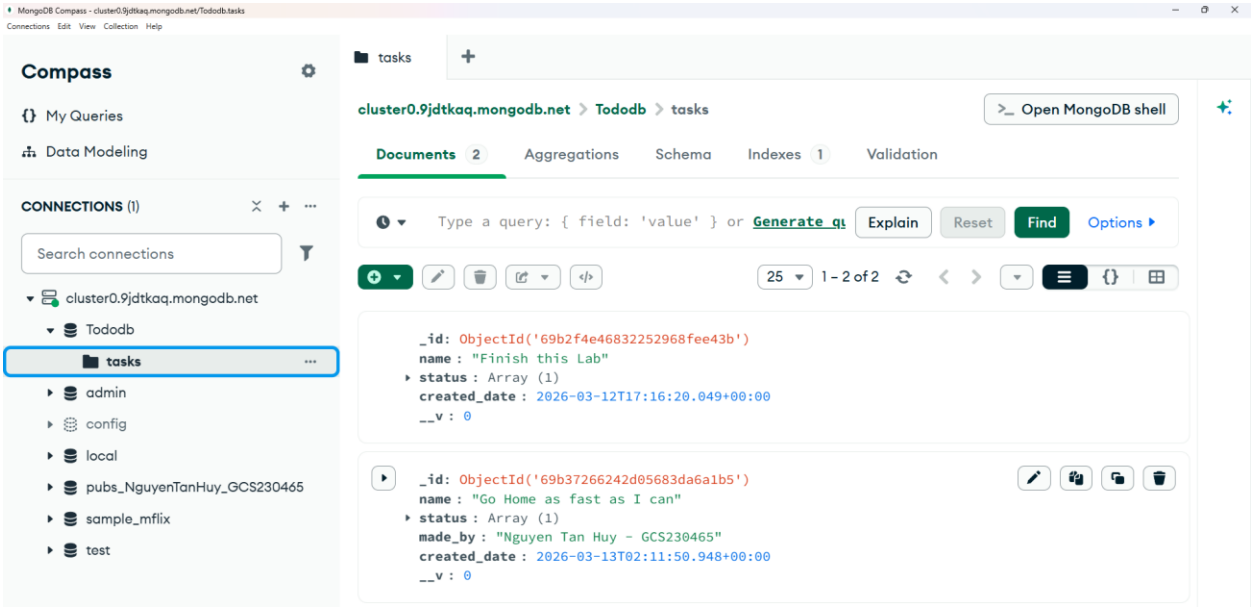
Params Authorization Headers (4) Body Options Code

KEY	VALUE	DESCRIPTION
Key	Value	Description

Body Headers JSON Status: 200 OK Duration: 56 ms

[{"made_by":"Nguyen Tan Huy - GCS230465","_id":"69b2f4e4683225296 8fee43b","name":"Finish this Lab","status":["pending"],"created_date":"2026

Image 4: My data in my MongoDB



Reflection on RESTful API Development and Integration:

During this session, I explored the practical implementation of RESTful APIs using the Node.js environment, specifically focusing on the architectural separation of models, routes, and controllers. While my previous experience in course AMD201 involved utilizing ASP.NET Core, I found the transition to a JavaScript-based stack enlightening. Although the syntax and middleware configurations—such as using body-parser to handle HTTP request bodies—differ from the C# ecosystem, the underlying principles of the Scrum framework and organized system design remain consistent.

The implementation process highlighted the critical importance of dependency management. Initial execution errors served as a valuable reminder to ensure all necessary packages, such as Express and Mongoose, are correctly installed and initialized. Witnessing the server successfully initialize on port 3000 was a significant milestone in validating the environment setup.

Furthermore, I gained proficiency in using Postcode (an API client similar to Postman) to test CRUD operations. Successfully performing a POST request to create a "task" resource and subsequently retrieving it via a GET request provided tangible evidence of the seamless communication between the client, the API, and the MongoDB database. Verifying the persistent data directly within the database confirmed my understanding of APIs as a robust mechanism for connecting disparate software components. This exercise has solidified my technical skills in full-stack development and reinforced the value of modular software design.